

# Implementación de Autenticación y Autorización para Sistema de Gestión de Tareas

## 1. Introducción

El presente informe documenta el desarrollo de una solución profesional para el desafío práctico titulado “*Implementación Básica de Autenticación y Autorización en una Aplicación Web*”. El objetivo es garantizar que el sistema no solo cumpla con los requisitos funcionales propuestos, sino que también se adhiera a los principios de seguridad, mantenibilidad y escalabilidad exigidos en un entorno real de producción.

## 2. Arquitectura General del Sistema

La solución propuesta se estructura bajo una arquitectura cliente-servidor con los siguientes componentes fundamentales:

- **Frontend (cliente web):** Interfaz gráfica para registro, inicio de sesión y operaciones sobre tareas.
- **Backend (API RESTful con Node.js + Express.js):** Responsable de la lógica de negocio, autenticación, autorización y persistencia.
- **Base de datos (MongoDB o MySQL):** Almacenamiento de usuarios, roles, contraseñas hasheadas y tareas.

La comunicación entre cliente y servidor se basa en HTTP y manejo de sesiones (JWT o sesiones de servidor), asegurando una separación clara de responsabilidades.

## 3. Seguridad de Autenticación

### 3.1 Registro de Usuario

- Se implementó un endpoint `/register` que recibe `username` y `password`.
- La contraseña es hasheada usando `bcrypt` con un mínimo de 10 salt rounds antes de ser almacenada.
- El rol asignado por defecto es `Usuario Común`, garantizando el principio de menor privilegio.

### 3.2 Inicio de Sesión

- El endpoint `/login` autentica al usuario mediante comparación del hash de contraseña.
- Al autenticarse correctamente, se genera un **JWT firmado** (o se crea una sesión en el servidor).
- El token incluye el rol del usuario y su ID, y tiene una expiración predefinida (por ejemplo, 30 minutos).

## 4. Control de Acceso Basado en Roles

### 4.1 Roles y Privilegios

- **Administrador:** Ver, agregar, editar y eliminar tareas.
- **Usuario Común:** Ver y agregar tareas solamente.

### 4.2 Middleware de Autenticación

- Verifica la validez y firma del token en cada solicitud protegida.
- Si el token es inválido o ha expirado, se retorna un error `401 Unauthorized`.

### 4.3 Middleware de Autorización

- Inspecciona el rol del usuario extraído del token y permite o restringe el acceso a rutas como `/tasks/delete`, `/tasks/update`, etc.
- Retorna error `403 Forbidden` en caso de insuficientes privilegios.

## 5. Funcionalidades de Tareas

### 5.1 Endpoints Implementados

- `GET /tasks`: Ver tareas (ambos roles).
- `POST /tasks`: Agregar tareas (ambos roles).
- `PUT /tasks/:id`: Solo Administradores.
- `DELETE /tasks/:id`: Solo Administradores.

Todos los endpoints están protegidos por middleware de autenticación y autorización.

## 6. Cierre de Sesión

- La sesión es cerrada en el cliente mediante eliminación del token (almacenado en `localStorage` o `sessionStorage`).
- En caso de implementación con sesiones de servidor, se destruye la sesión mediante `req.session.destroy()`.

## 7. Validación y Seguridad Adicional

- Todas las entradas del usuario son sanitizadas para prevenir ataques de inyección SQL o XSS.
- Se implementan validadores en los modelos de tareas y usuarios.
- Se sugiere una futura extensión con MFA (doble autenticación) y refresh tokens para renovación segura del JWT.

## 8. Herramientas Utilizadas

- **Node.js + Express.js:** Servidor backend y middleware.
- **bcrypt:** Hasheo seguro de contraseñas.
- **jsonwebtoken:** Firma y verificación de tokens JWT.
- **MongoDB / MySQL:** Almacenamiento persistente de entidades.
- **Postman / Insomnia:** Pruebas automatizadas de la API.

## 9. Pruebas Realizadas

- Usuario Común intentando editar tareas: acceso denegado (403).
- Administrador eliminando tarea: acceso concedido.
- Token inválido: denegado con error 401.
- Flujo completo de registro, login, uso de recursos y logout validado manual y automáticamente.

## 10. Conclusión

La solución propuesta no solo cumple con los requerimientos del ejercicio práctico, sino que eleva su implementación a un estándar profesional. La separación de responsabilidades, el uso de middleware para autenticación y autorización, el almacenamiento seguro de contraseñas y la protección de rutas garantizan un sistema robusto, extensible y seguro. Se recomienda utilizar esta base como punto de partida para proyectos de mayor complejidad, incluyendo escalado horizontal, autenticación federada y sistemas de autorización más granulares basados en permisos.